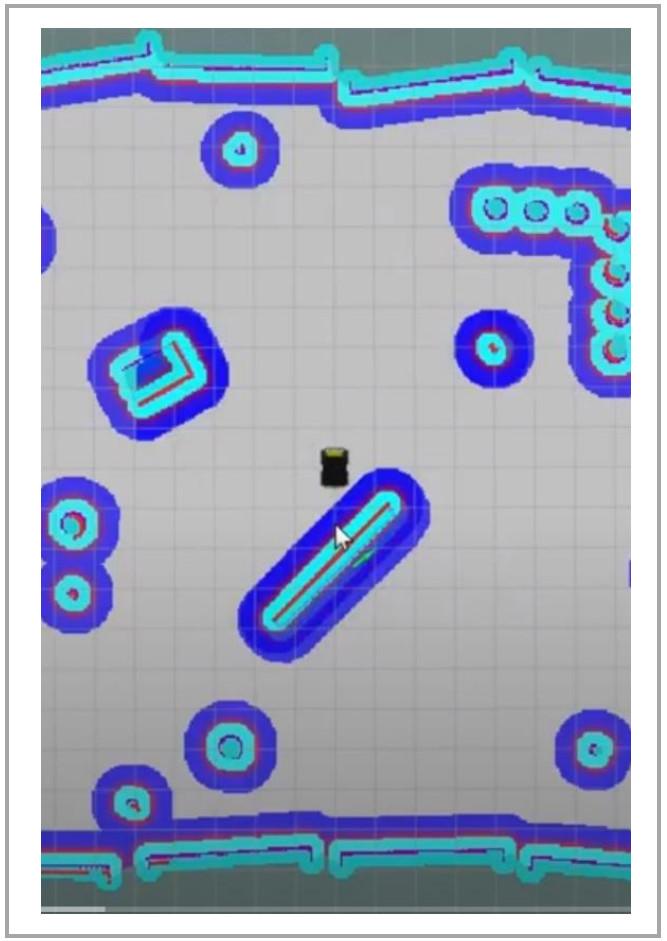
# Laboratoire 1

## GRO830 – Programmation robotique

Louis Petit, ing., Ph.D.
Professeur

# PLAN DE LA PRÉSENTATION



1 **CARTOGRAPHIE ET NAVIGATION**

Utilisation de RTAB-Map, Rviz et Gazebo

2 **CAPTEURS**

Détection de couleurs par caméra

3 **ARCHITECTURE DE CONTRÔLE**

Arbitration, comportements, suivi de trajectoire

4 **QUESTIONS ET RÉPONSES**



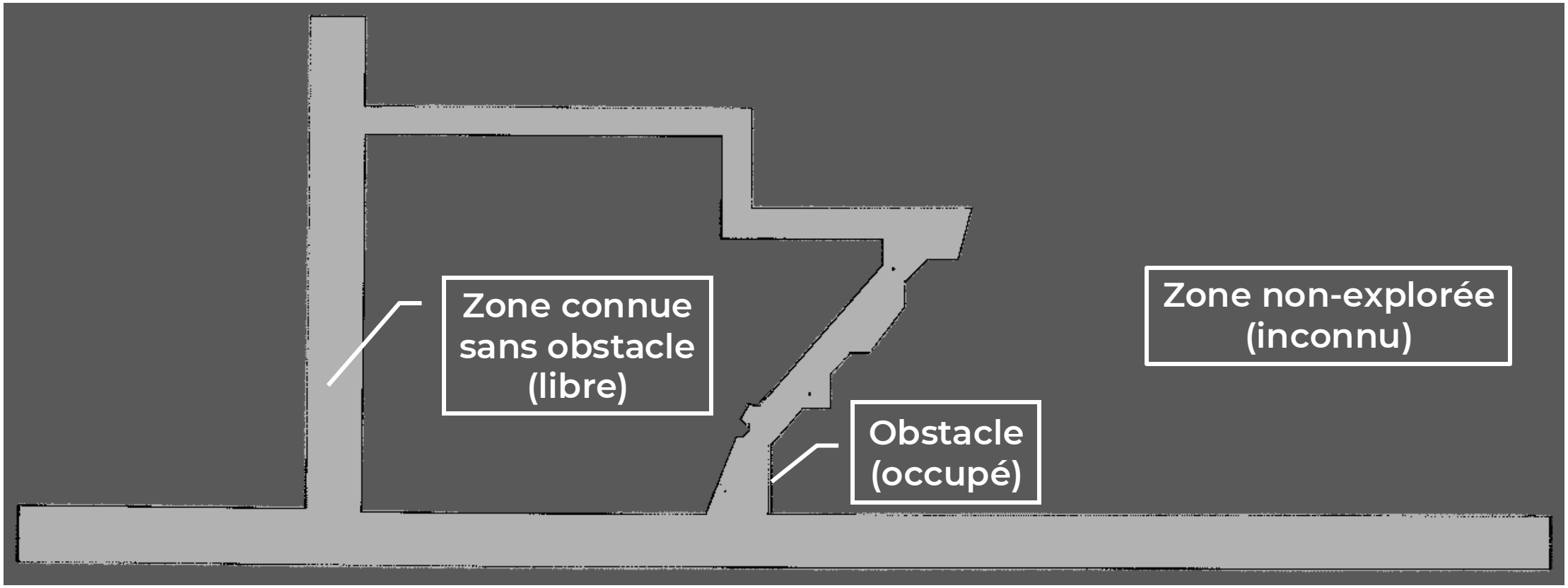
Source : Tinker Twins

# Cartographie et navigation

## Utilisation de RTAB-Map, Rviz et Gazebo

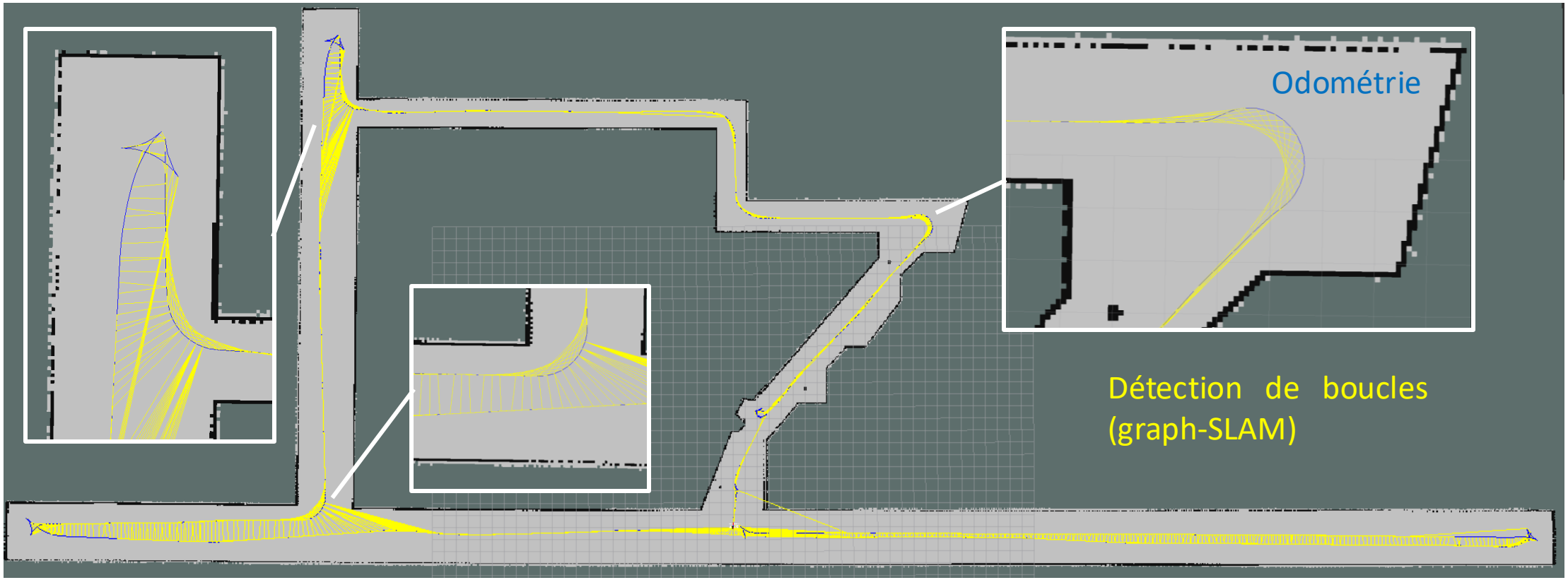
# CARTOGRAPHIE

## Exercice no1a



# CARTOGRAPHIE

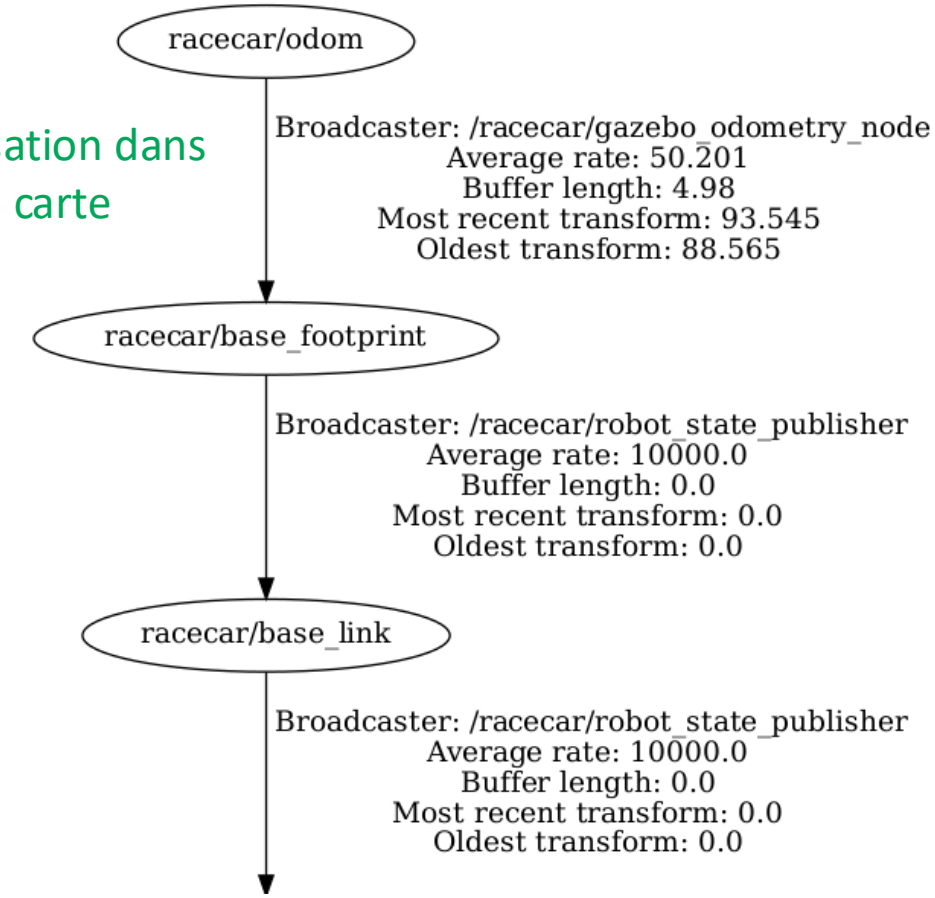
## Exercice no1a



# LOCALISATION

## Exercice no1b

Localisation dans la carte




**map :** l'origine de la carte

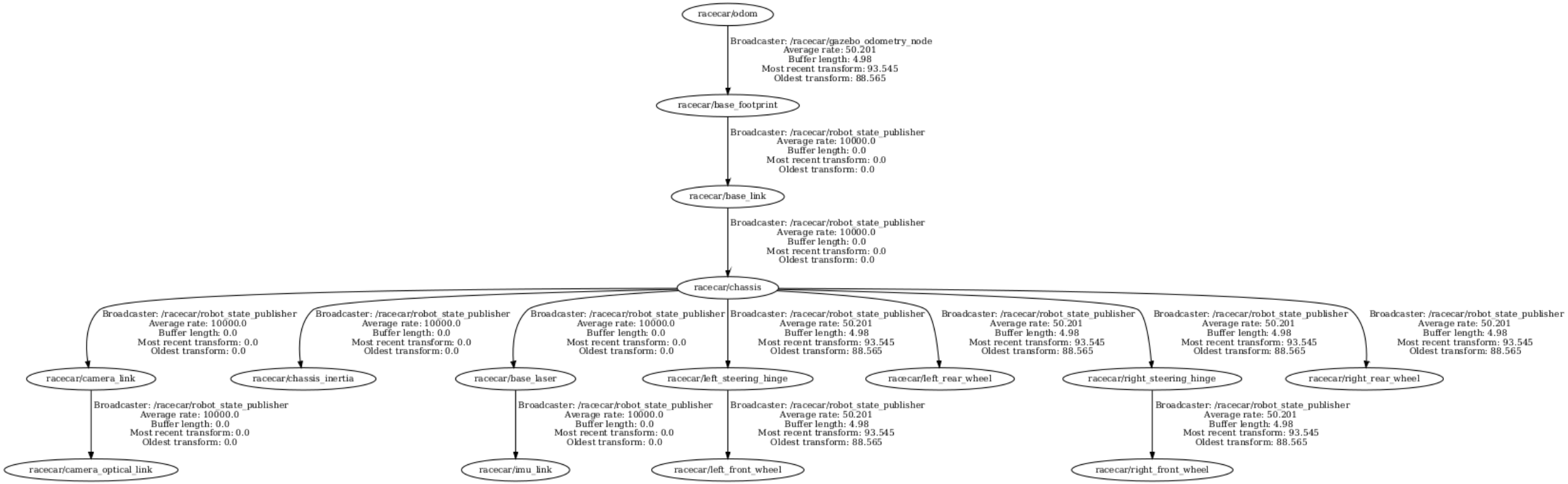
**odom** : la position initiale du robot dans la carte

**base_footprint** : projection 2D de la base du robot au sol

**base_link** : l'origine de la base du robot

# LOCALISATION

## Exercice no1b

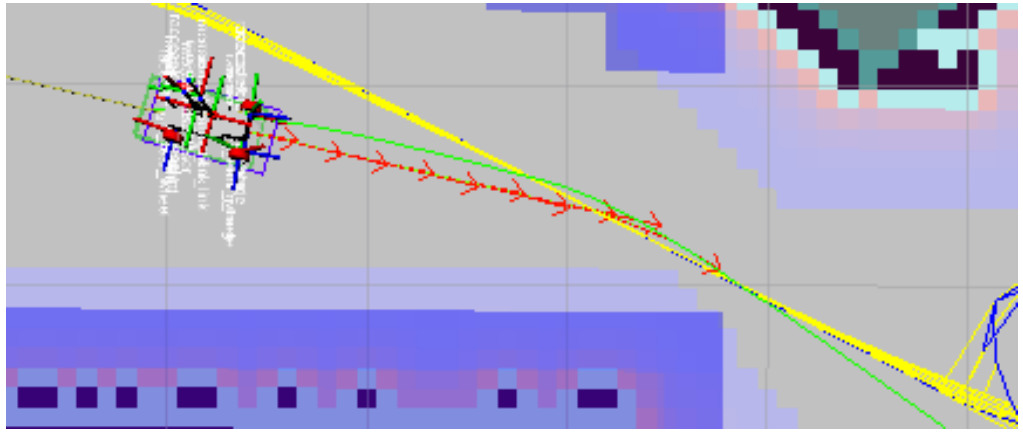


# NAVIGATION

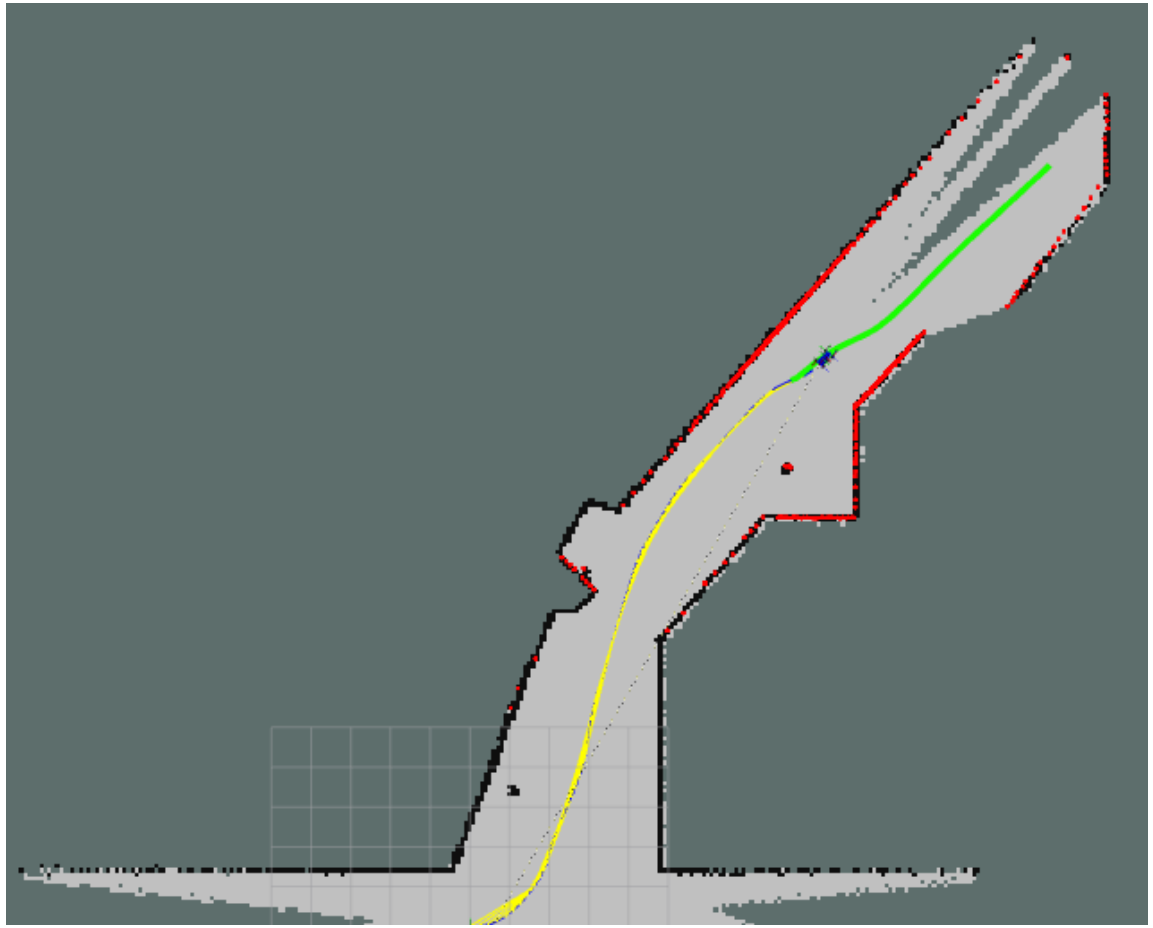
## Exercice no1c

**Vert** : trajectoire globale (par exemple avec l'algorithme A*)

**Rouge** (anciennement dans MoveBase) : trajectoire locale (par exemple avec un algorithme de champ de potentiel)



MoveBase



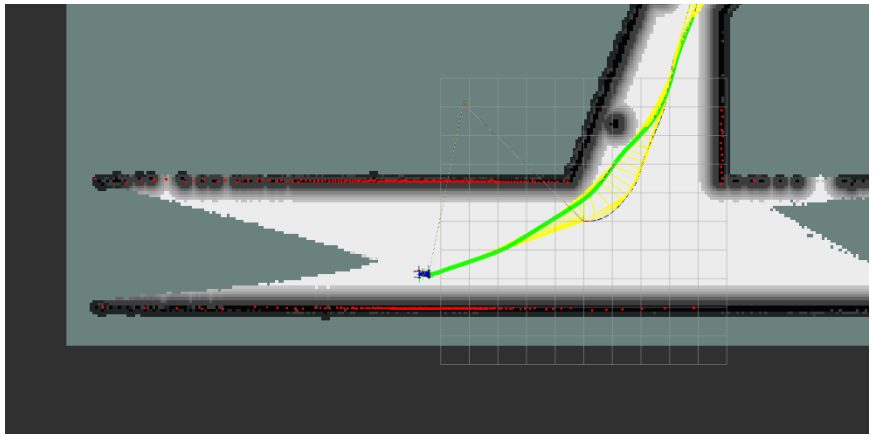
Nav2

# NAVIGATION

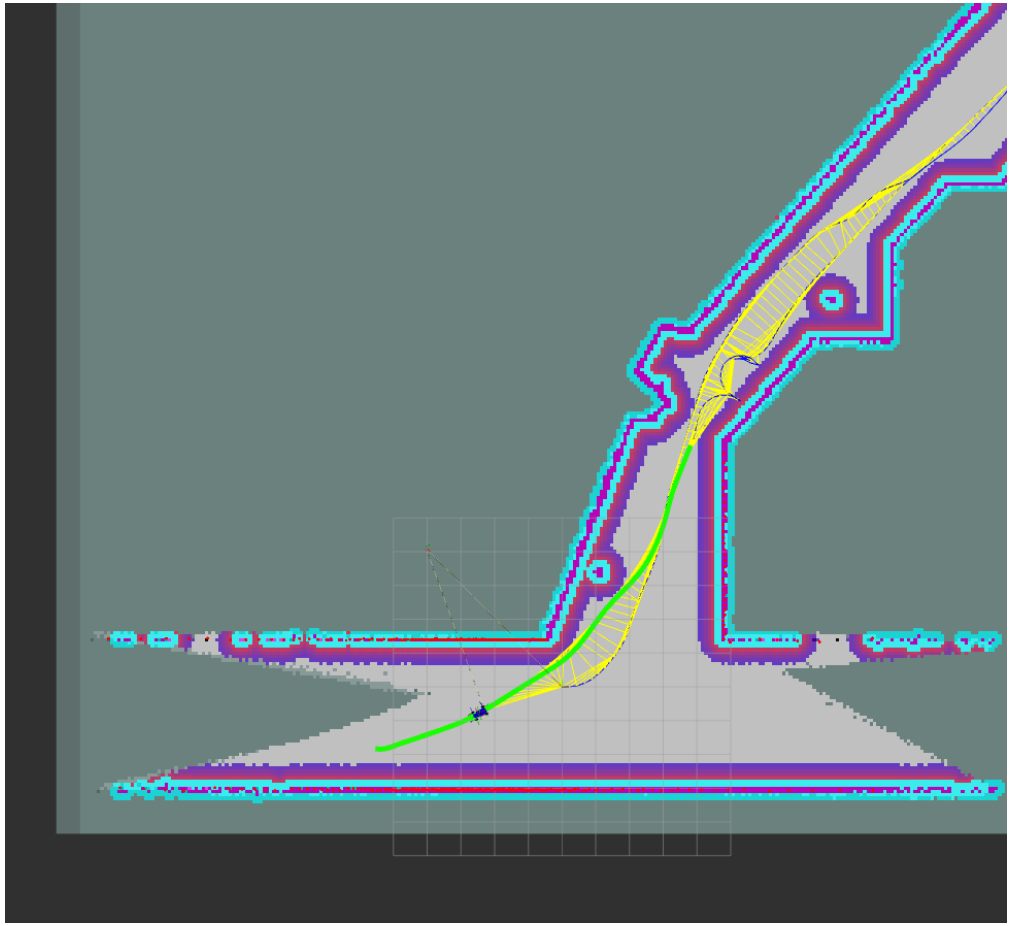
## Exercice no1c

**Bleu/Mauve** : carte d'obstacles et inflation

**Vert** : trajectoire en-dehors de la zone d'inflation si possible



Color scheme: map



Color scheme: costmap

# CARTE D'OBSTACLES

Exercice no2a

## nav_msgs/GetMap Service

**File:** nav_msgs/GetMap.srv

### Raw Message Definition

```
# Get the map as a nav_msgs/OccupancyGrid
---
nav_msgs/OccupancyGrid map
```

### Compact Message Definition

```
---
nav_msgs/OccupancyGrid map
```

## nav_msgs/OccupancyGrid Message

**File:** nav_msgs/OccupancyGrid.msg

### Raw Message Definition

```
# This represents a 2-D grid map, in which each cell represents the probability of
# occupancy.

Header header

#MetaData for the map
MapMetaData info

# The map data, in row-major order, starting with (0,0).  Occupancy
# probabilities are in the range [0,100].  Unknown is -1.
int8[] data
```

Source : ROS, nav_msgs

# CARTE D'OBSTACLES

Exercice no2a

## nav_msgs/MapMetaData Message

**File:** nav_msgs/MapMetaData.msg

### Raw Message Definition

```
# This hold basic information about the characterists of the OccupancyGrid

# The time at which the map was loaded
time map_load_time
# The map resolution [m/cell]
float32 resolution
# Map width [cells]
uint32 width
# Map height [cells]
uint32 height
# The origin of the map [m, m, rad].  This is the real-world pose of the
# cell (0,0) in the map.
geometry_msgs/Pose origin
```

# CARTE D'OBSTACLES

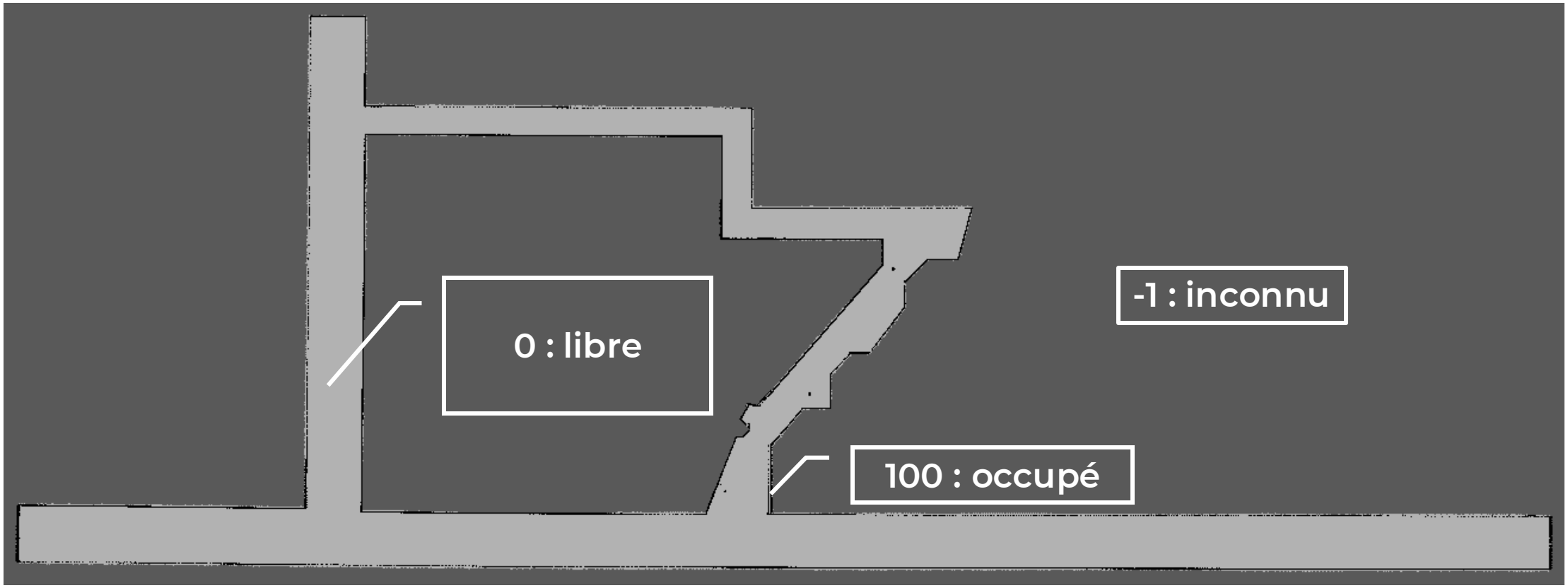
Exercice no2a

Pour connaître la case de la carte associée à une position :

(pose - origin ) / resolution

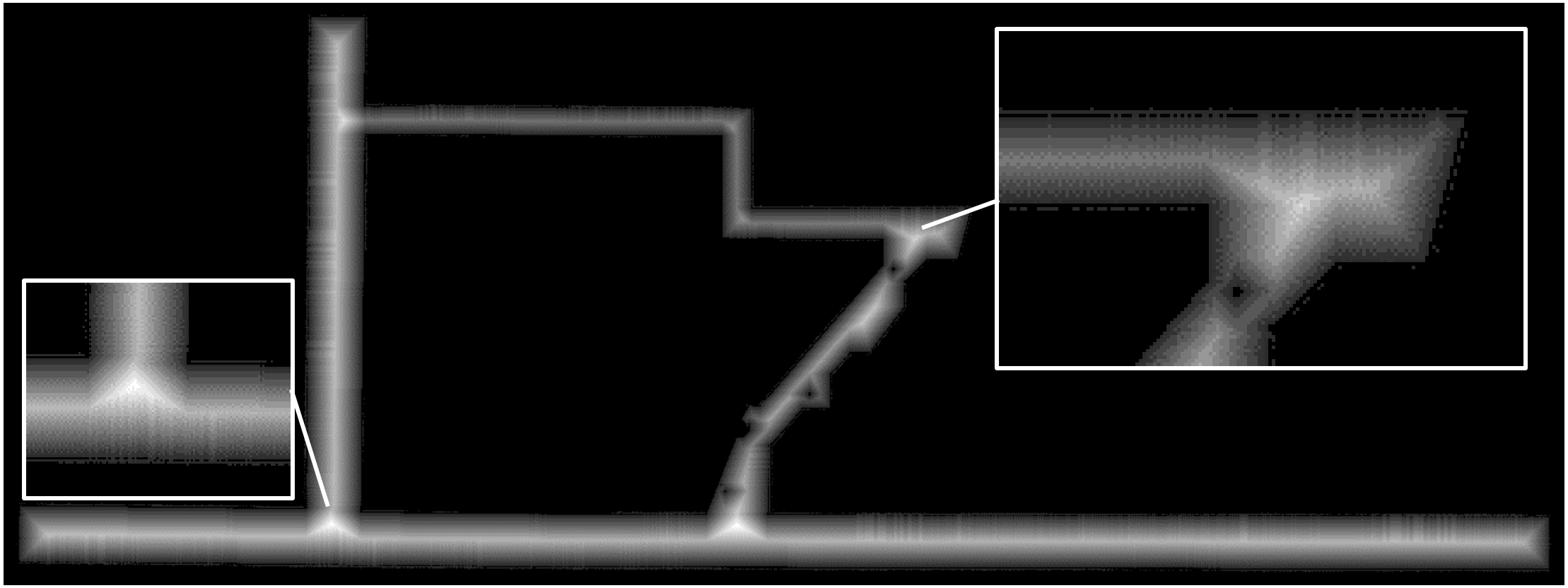
# CARTE D'OBSTACLES

Exercice no2b



# CARTE D'OBSTACLES

Exercice no2b



# CARTE D'OBSTACLES

Exercice no2b

```
while 0 in mapOfWorld:
  a = a + 1
  for iRow in range(nRows):
    for iCol in range(nCols):
      if mapOfWorld[iRow][iCol] == a:
        # process 4 cells window here
        if (use8CellWindow):
          # process 4 remaining cells to complete 8 cells window
```

# CARTE D'OBSTACLES

## Exercice no2b

Connectivité par 4

```
if (iRow > 0):    # Ne pas considérer les voisins en-dehors de la carte
  if (mapOfWorld[iRow-1][iCol] == 0) : mapOfWorld[iRow-1][iCol] = a + 1
if (iRow < nRows-1):
  if (mapOfWorld[iRow+1][iCol] == 0) : mapOfWorld[iRow+1][iCol] = a + 1
if (iCol > 0):
  if (mapOfWorld[iRow][iCol-1] == 0) : mapOfWorld[iRow][iCol-1] = a + 1
if (iCol < nCols-1):
  if (mapOfWorld[iRow][iCol+1] == 0) : mapOfWorld[iRow][iCol+1] = a + 1
```

# CARTE D'OBSTACLES

## Exercice no2b

Connectivité par 8

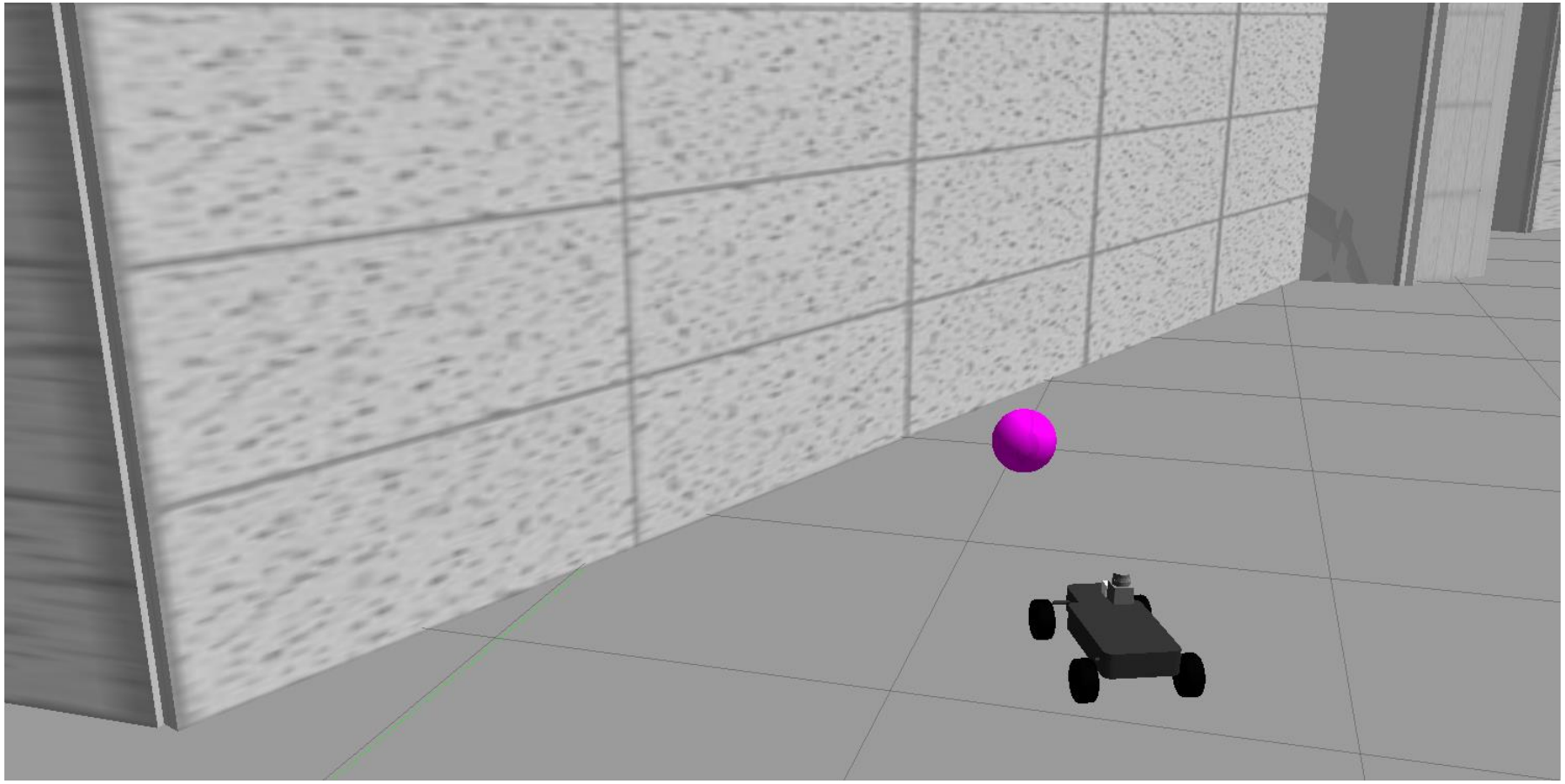
```
if (iRow > 0) and (iCol > 0):
  if (mapOfWorld[iRow-1][iCol-1]) == 0 : mapOfWorld[iRow-1][iCol-1] = a + 1
if (iRow > 0) and (iCol < nCols-1):
  if (mapOfWorld[iRow-1][iCol+1]) == 0 : mapOfWorld[iRow-1][iCol+1] = a + 1
if (iRow < nRows-1) and (iCol > 0):
  if (mapOfWorld[iRow+1][iCol-1]) == 0 : mapOfWorld[iRow+1][iCol-1] = a + 1
if (iRow < nRows-1) and (iCol < nCols-1):
  if (mapOfWorld[iRow+1][iCol+1]) == 0 : mapOfWorld[iRow+1][iCol+1] = a + 1
```

# Capteurs

## Détection de couleurs par caméra

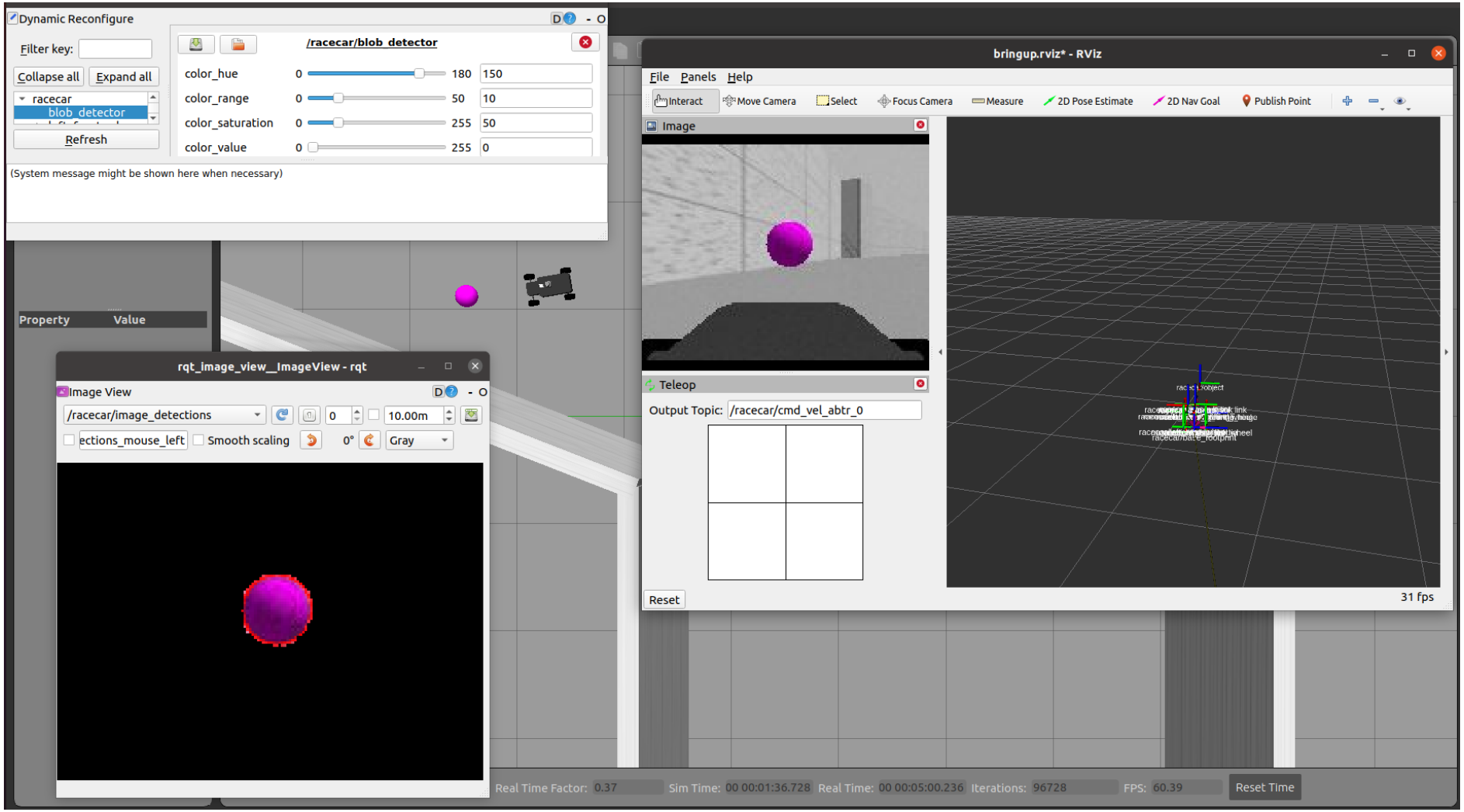
# DÉTECTION DE BLOB

## Exercice no 3a



# DÉTECTION DE BLOB

## Exercice no 3a



# DÉTECTION DE BLOB

## Exercice no 3a

**Il y a du bruit provenant des capteurs!**

```
[INFO] [1696176370.368933, 273.383000]: Object detected at [3.143007,0.916429] in racecar/odom frame! Distance and direction from robot: 0.778287m -0.770302deg.
[INFO] [1696176370.610083, 273.490000]: Object detected at [3.159364,0.919271] in racecar/odom frame! Distance and direction from robot: 0.794890m -0.780189deg.
[INFO] [1696176370.827872, 273.581000]: Object detected at [3.160870,0.919833] in racecar/odom frame! Distance and direction from robot: 0.796471m -0.759804deg.
[INFO] [1696176371.071954, 273.684000]: Object detected at [3.155996,0.918510] in racecar/odom frame! Distance and direction from robot: 0.791440m -0.790684deg.
[INFO] [1696176371.259310, 273.789000]: Object detected at [3.165853,0.920220] in racecar/odom frame! Distance and direction from robot: 0.801443m -0.796506deg.
[INFO] [1696176371.508131, 273.883000]: Object detected at [3.164105,0.920231] in racecar/odom frame! Distance and direction from robot: 0.799726m -0.773347deg.
[INFO] [1696176371.794902, 273.984000]: Object detected at [3.153846,0.918245] in racecar/odom frame! Distance and direction from robot: 0.789277m -0.781663deg.
```

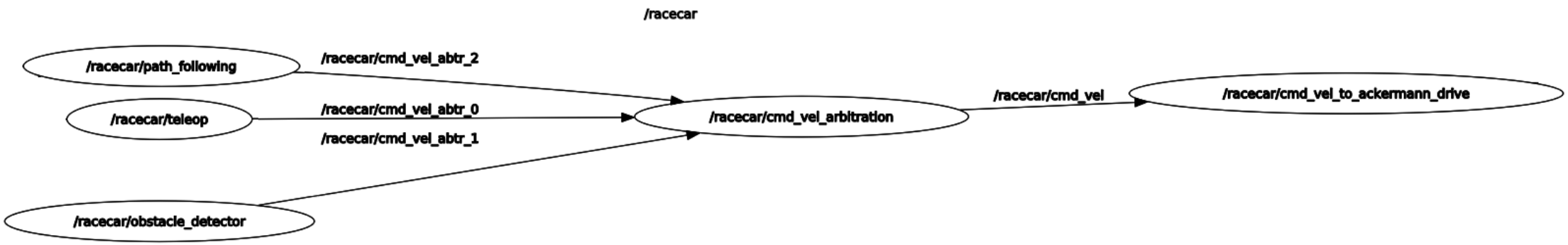
# Architecture de contrôle

## Arbitration, comportements, suivi de trajectoire

# ARCHITECTURE DE CONTRÔLE

## Exercice no4a

Prioriser les commandes
entre différents modules



# ARCHITECTURE DE CONTRÔLE

Exercice no4a

```
self._cmd_vel_subs = []
for i in range(8):
    sub = self.create_subscription(Twist, f'cmd_vel_abtr_{i}', lambda msg, priority=i: self.cmd_vel_callback(msg, priority), 1)
    self._cmd_vel_subs.append(sub)
```

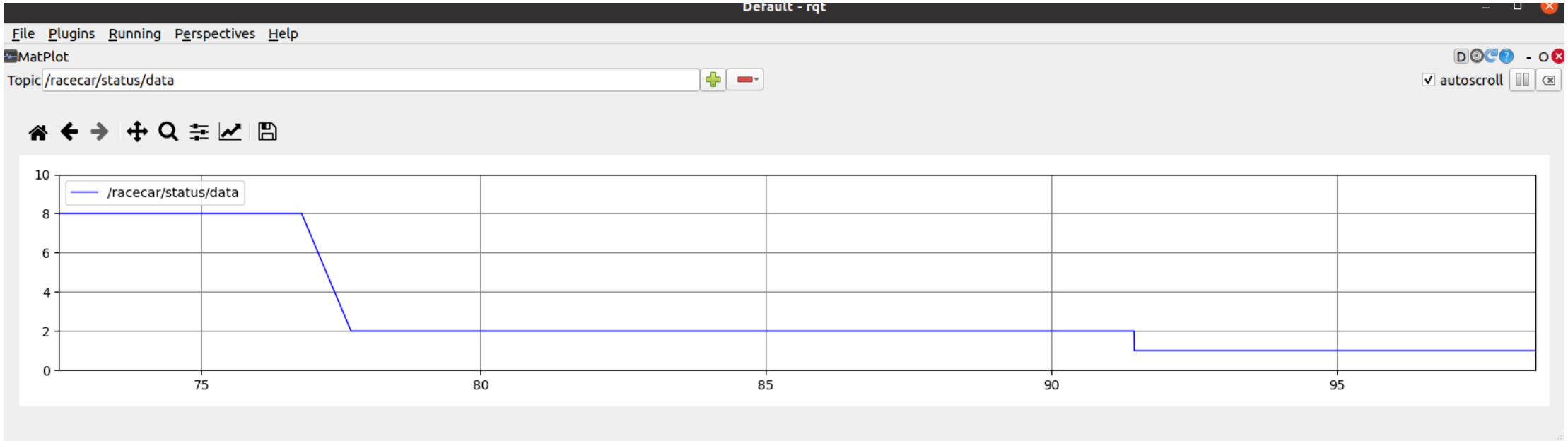
```
<node pkg="racecar_behaviors" type="path_following.py" name="path_following">
  <param name="max_speed" value="1"/>
  <remap from="cmd_vel" to="cmd_vel_abtr_2"/>
  <remap from="scan"    to="scan"/>
  <remap from="odom"    to="odometry/filtered"/>
</node>

<node pkg="racecar_behaviors" type="obstacle_detector.py" name="obstacle_detector">
  <remap from="cmd_vel" to="cmd_vel_abtr_1"/>
  <remap from="scan"    to="scan"/>
</node>
```

Source : behaviors.launch

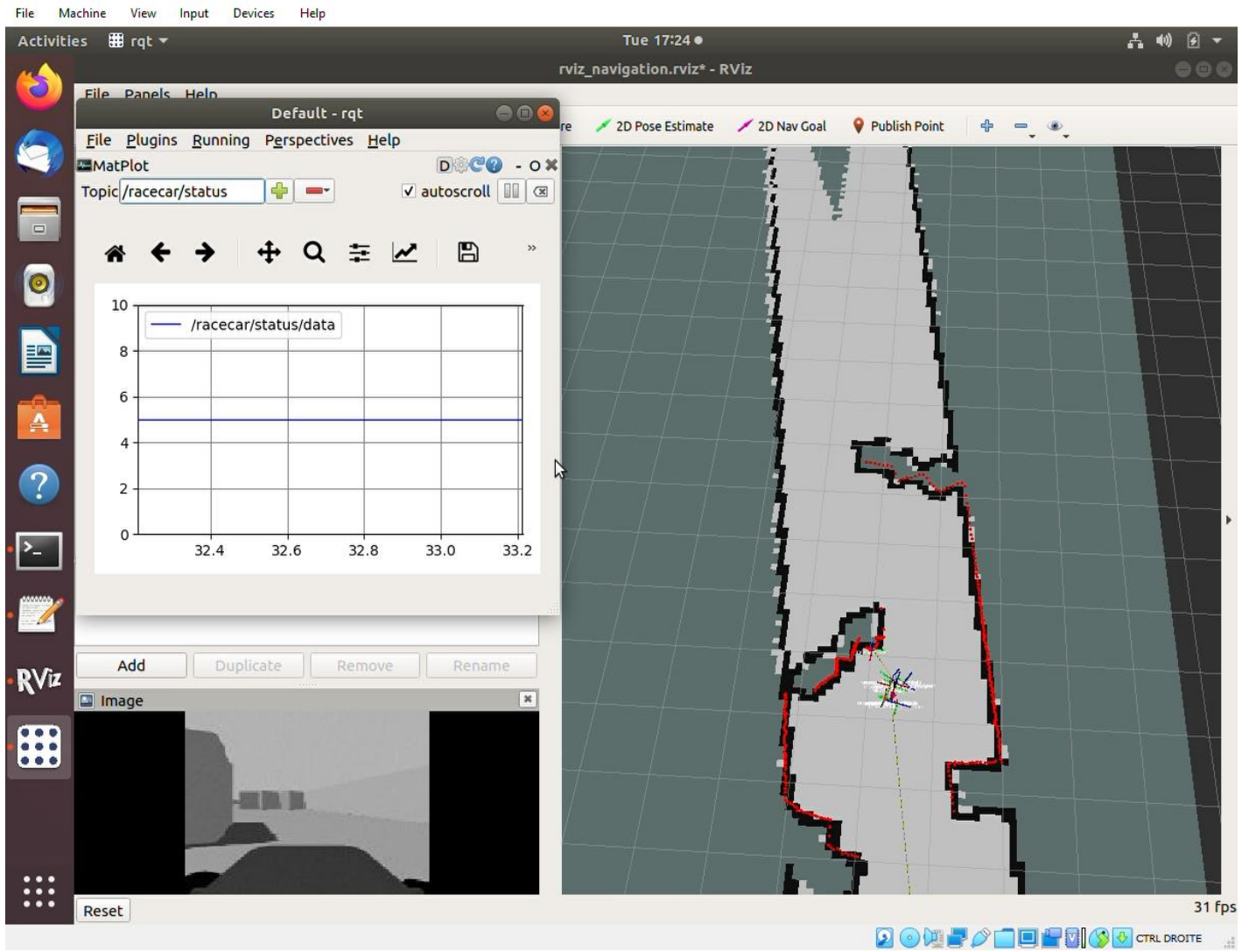
# ARCHITECTURE DE CONTRÔLE

## Exercice no4a



# ÉVITEMENT D'OBSTACLES

## Exercice no4b

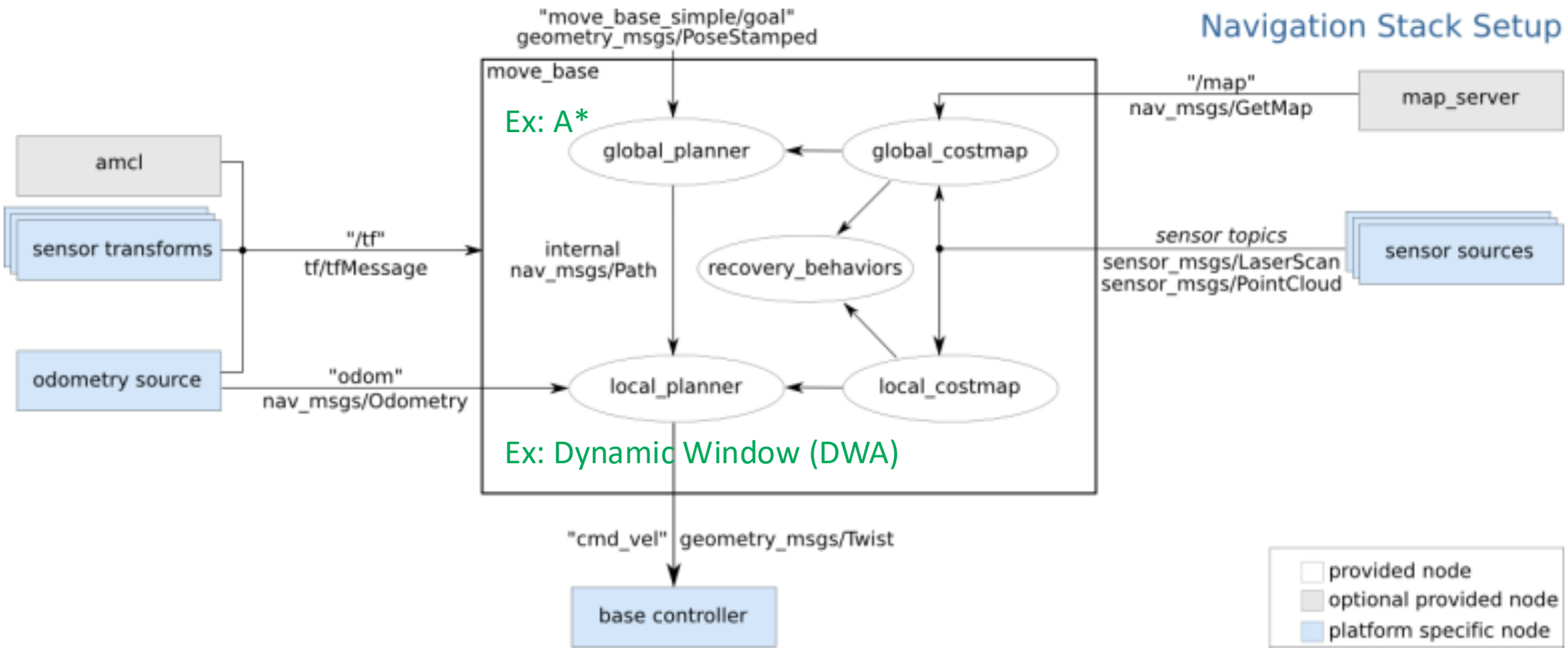



Source : ROS, move_base

# ÉVITEMENT D'OBSTACLES

Exercice no4b

*move_base* avant *nav2*…

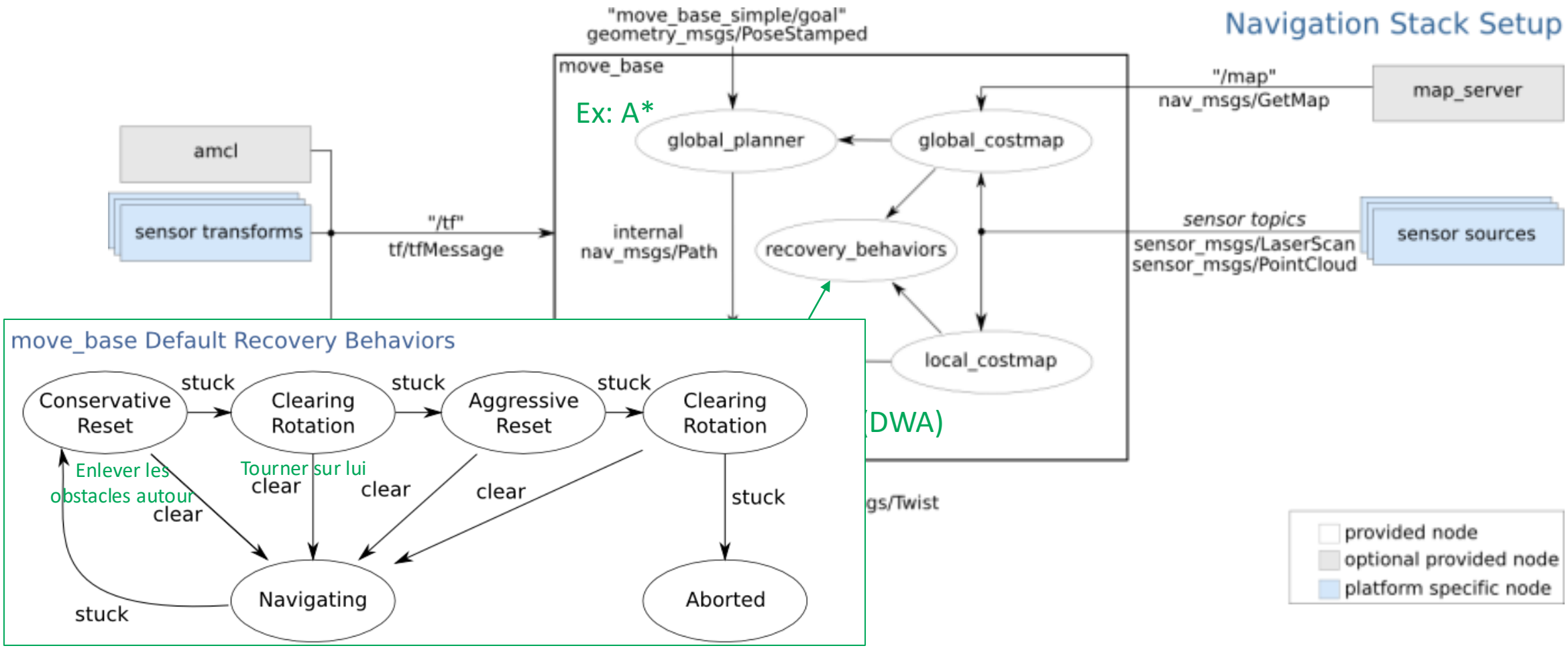



Source : ROS, move_base

# ÉVITEMENT D'OBSTACLES

Exercice no4b

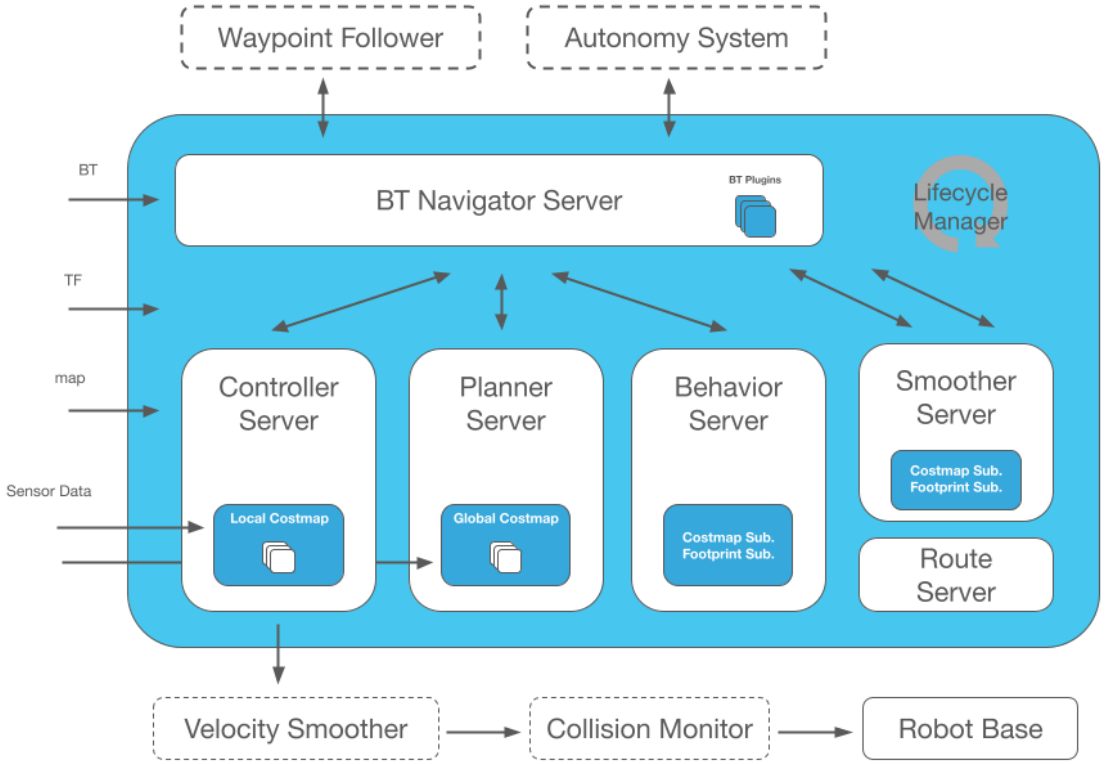
*move_base* avant *nav2*…

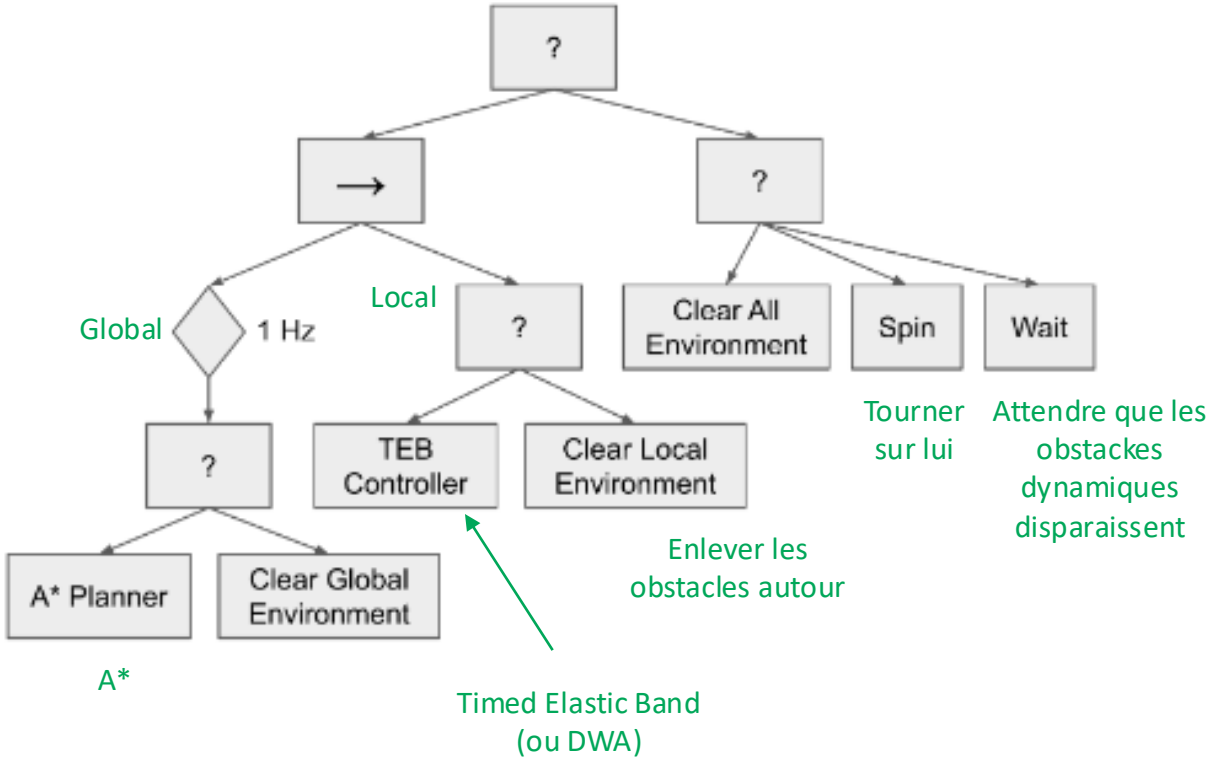


# ÉVITEMENT D'OBSTACLES

## Exercice no4b

*Behavior tree* pour l'arbitration



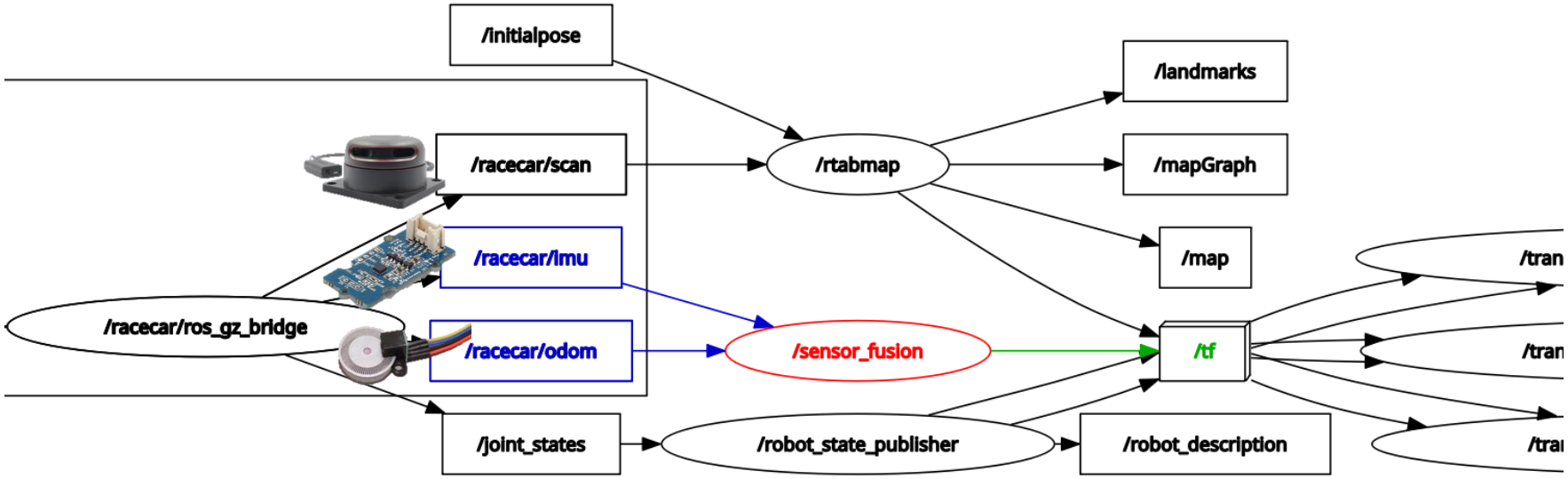



# FUSION DE CAPTEURS

## Exercice no5


```
header:
  stamp:
    sec: 670
    nanosec: 136000000
  frame_id: racecar/odom/filtered
child_frame_id: racecar/base_footprint
pose:
  pose:
    position:
      x: 1.6771113647486964
      y: 2.2385918177765722
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.5463736725251606
      w: 0.8375415273115528
  covariance:
  - 33.50715958378768
  - -9.178074151128985e-07
  - 0.0
  - 0.0
  - 0.0
  - 5.0274780239691455e-26
  - -9.178074151110215e-07
  - 33.5071586420771
  - 0.0
  - 0.0
  - 0.0
  - -2.2134870085563425e-26
  - 0.0
  - 0.0
  - 101386170630.86455
  - 2564427.155615283
  - -81415.71093523684
  - 0.0
  - 0.0
  - 0.0
  - 2564427.155615237
  - 531972.840018299
  - 1.824929096727601e-15
  - 0.0
```

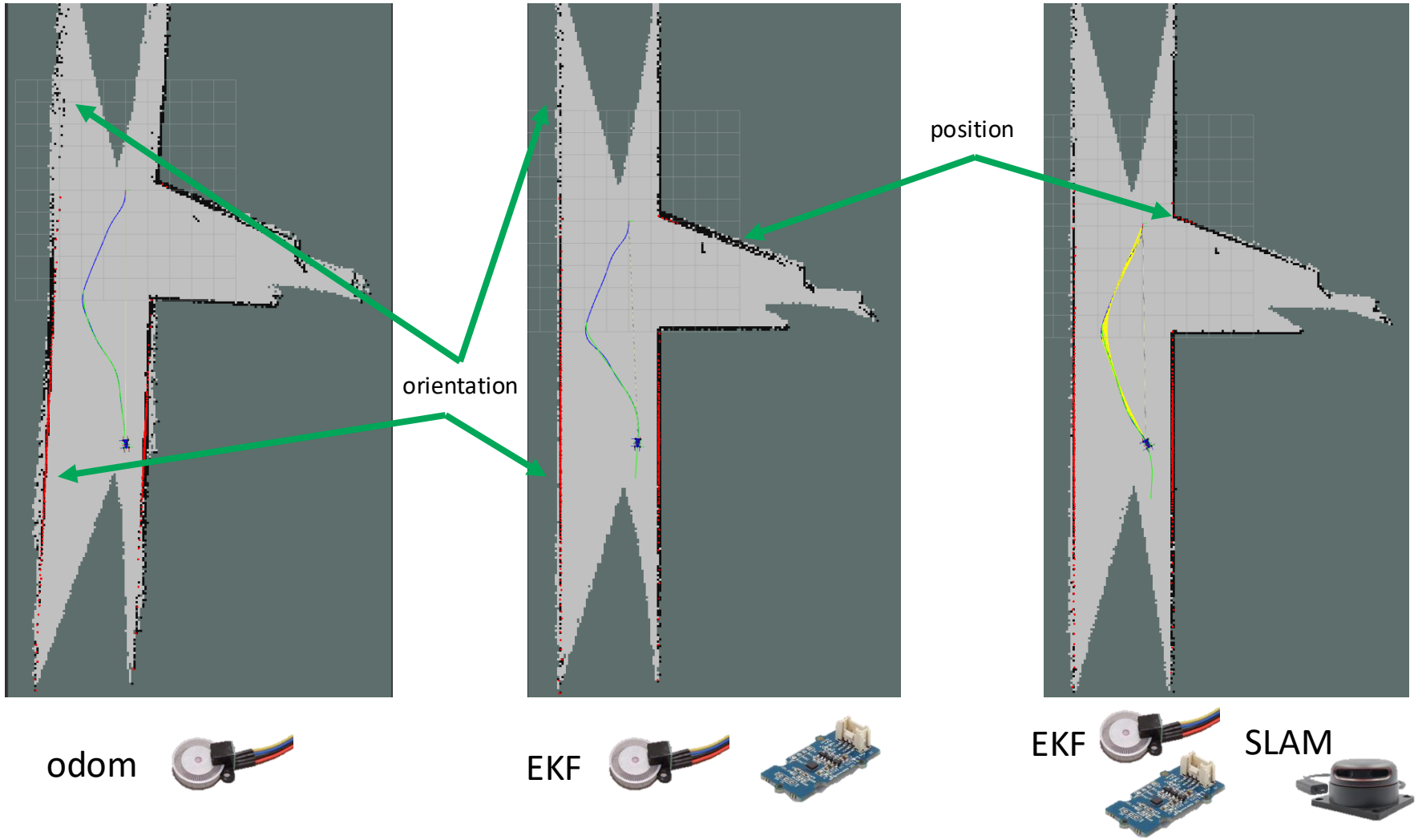

racecar/odom/filtered




rqt_graph

# FUSION DE CAPTEURS

## Exercice no5




Source : ROS, move_base

# Merci à tous!

Des questions?